EVPN/VXLAN Leaf-Spine Fabric

The Complete Teaching Guide

FRRouting + Containerlab: Beginner-to-Advanced

April 1, 2026

Abstract

This document is an exhaustive beginner-to-advanced technical reference for building, understanding, and troubleshooting an EVPN/VXLAN data center fabric using FRRouting and Containerlab. It covers every architectural concept from leaf-spine topology through Linux kernel dataplane mechanics, walks through a real lab repository file-by-file, provides end-to-end packet traces, troubleshooting playbooks, interview preparation, and a curated reference library. Whether you hold a CCNA and want to break into data center networking or you are debugging RMAC resolution failures in production, this guide meets you where you are.

Contents

1 Executive Overview **1**

2 CCNA-to-DC Bridge: Prerequisite Concepts **1**

3 Concept-by-Concept Deep Dive **2**

3.1 Leaf-Spine Topology and the Clos Fabric 2
3.2 Underlay versus Overlay: Why the Separation Exists 3
3.3 Why eBGP Replaced OSPF as the DC Underlay 3
3.4 VXLAN Encapsulation: L2 Frames Riding L3 Packets 4
3.5 VTEP: The Tunnel Endpoint on Every Leaf 4
3.6 VNI, L2VNI, and L3VNI: The Two Types of Virtual Network 5
3.7 L2VNI Bridging: Flood-and-Learn versus EVPN 5
3.8 L3VNI and Symmetric IRB: Routing Across Subnets 6
3.9 EVPN: The Control Plane that Replaced Flooding 6
3.10 EVPN Route Types: Type-2 and Type-5 in Detail 6
3.11 Route Distinguisher and Route Target 7
3.12 Anycast Gateway: Every Leaf Is the Default Gateway 7
3.13 VRF: Isolated Routing Tables for Multi-Tenancy 7
3.14 ARP Suppression: Eliminating Broadcast Floods 8
3.15 BFD: Sub-Second Failure Detection 8
3.16 Linux Bridge, FDB, and Neighbor Table 8
3.17 FRRouting Architecture: Zebra and the Daemon Model 9
3.18 Kernel Routing Tables and rp_filter 9
3.19 Containerlab: Networking Labs in Containers 9

4 Repository File Walkthrough **10**
4.1 `vxlan-static.clab.yml`: The 6-Node Topology . . . 10
4.2 `configs/daemons`: Which FRR Processes Run . . . 10
4.3 `configs/leaf1/frr.conf`: The Leaf BGP + EVPN Configuration . . . 11
4.4 `configs/sp1/frr.conf` and `configs/sp2/frr.conf`: Spine as EVPN Relay 12
4.5 `configs/leaf1/init.sh`: Linux Dataplane Setup (With Dead Code Bug) 13
4.6 Host Init Scripts: Simple IP Assignment . . . 14
4.7 `configs/vtysh.conf`: Empty File . . . 14
4.8 Generated Artifacts: `clab-underlay/` and `clab-vxlan-static/` . . . 14
4.9 `underlay.clab.yml`: Simpler Pre-EVPN Topology . . . 15

5 End-to-End Packet Walkthroughs **15**
5.1 h1 Pings h2: Inter-Subnet Routing via Symmetric IRB . . . 15
5.2 Type-2 Route Learning Flow . . . 16
5.3 Type-5 Route Learning Flow . . . 16
5.4 BFD and Failure Convergence . . . 16

6 Control Plane versus Data Plane **17**

7 Verification and Troubleshooting Lab Guide **18**
7.1 FRR Verification Commands . . . 18
7.2 Linux Kernel Verification Commands . . . 20
7.3 Host-Level Reachability Tests . . . 21
7.4 Common Failure Patterns and Remediation . . . 22

8 Repository Weaknesses and Cleanup Opportunities **22**
8.1 Critical Bug: Dead Code After `exit 0` . . . 22
8.2 Empty `vtysh.conf` . . . 23
8.3 Generated Artifacts Committed to Repository . . . 23
8.4 Non-Standard Route Target Values . . . 23
8.5 L2VNI Relies Entirely on `advertise-all-vni` . . . 23
8.6 No Cross-Leaf L2 Stretching . . . 23
8.7 Summary of Issues . . . 24

9 Interview Preparation **24**
9.1 The 30-Second Pitch . . . 24
9.2 The 2-Minute Explanation . . . 24
9.3 The Deep Technical Explanation for a Network Engineer Interviewer . . . 25
9.4 Likely Interview Questions and Strong Answers . . . 25
9.5 What Is Most Technically Impressive . . . 26

10 Learning Roadmap **26**

11 Reference Library **28**
11.1 RFCs and Standards . . . 28
11.2 FRRouting Documentation . . . 29
11.3 Containerlab . . . 30
11.4 Vendor Design Guides . . . 30
11.5 Learning Resources and Courses . . . 31

11.6 Linux Networking Documentation 32
11.7 Related Open-Source Labs 32

1 Executive Overview

This teaching document deconstructs a 6-node EVPN/VXLAN lab built with FRRouting (FRR) containers orchestrated by Containerlab. The topology consists of **two spine switches (sp1, sp2)**, **two leaf switches (leaf1, leaf2)**, and **two Alpine Linux hosts (h1, h2)**. The fabric uses eBGP as the underlay routing protocol (AS 65101, 65102 for leaves; AS 65000 for spines), VXLAN for data-plane encapsulation, and EVPN (MP-BGP with AFI 25/SAFI 70) as the control plane for MAC/IP advertisement. Host h1 sits on subnet `192.168.101.0/24` behind leaf1, and h2 sits on `192.168.102.0/24` behind leaf2. Inter-subnet routing occurs exclusively through a **symmetric IRB model** using L3VNI 50100 within VRF `vrf-ten1` — there is no L2 stretching across leaves.

The lab demonstrates real-world DC fabric principles: every concept from ECMP load balancing through RMAC-based forwarding is exercised. The repo also contains **significant bugs** — most notably dead code after `exit 0` in init scripts — which provide excellent teaching moments about Linux shell scripting and operational rigor.

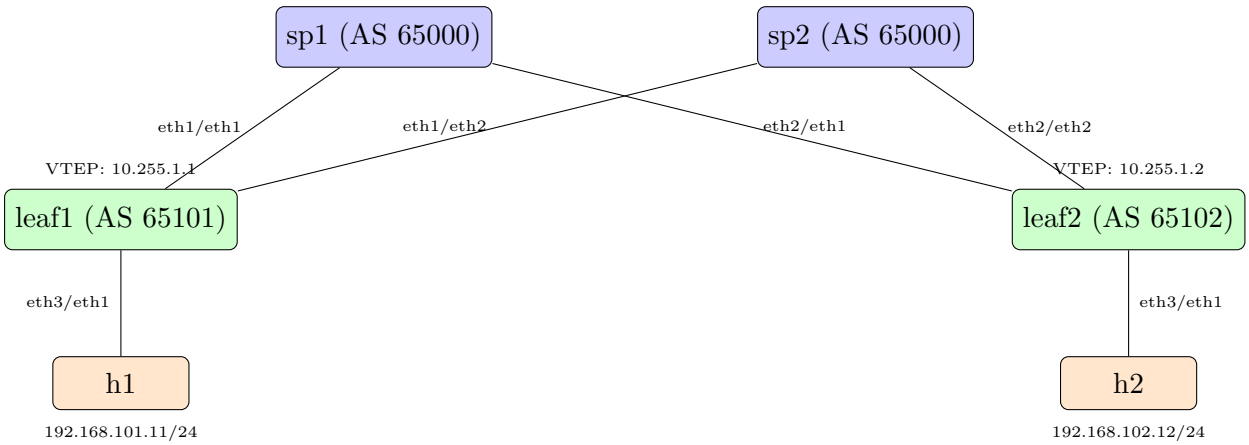

Figure 1: Lab topology: two spines, two leaves, two hosts. Full-mesh leaf-to-spine connectivity via eBGP. VXLAN tunnels between VTEP loopbacks.

2 CCNA-to-DC Bridge: Prerequisite Concepts

If you hold a CCNA, you already understand VLANs, STP, inter-VLAN routing, OSPF, and basic BGP. Data center networking takes these foundations and replaces the parts that do not scale. The three-tier model (access → aggregation → core) you learned was designed for north-south traffic — clients accessing servers through a hierarchy. Modern data centers are dominated by **east-west traffic** (server-to-server: microservices calling each other, distributed storage replication, VM migration). In three-tier, east-west traffic hairpins through aggregation or core layers, wasting bandwidth and adding latency. Spanning Tree Protocol blocks redundant links, leaving up to 50% of capacity unused.

The leaf-spine topology eliminates STP entirely by running **Layer 3 routing across the entire fabric** and using ECMP to utilize every link simultaneously. VLANs are replaced by VXLAN (scaling from 4,094 segments to **16.7 million**). OSPF is replaced by eBGP (per RFC 7938). Flood-and-learn MAC discovery is replaced by EVPN control-plane

learning. These are not exotic technologies — they are the standard architecture at every hyperscaler and most enterprise data centers built since 2015.

Table 1: Three-tier campus model versus leaf-spine data center model

Attribute	**Three-Tier Campus**	**Leaf-Spine DC**
Traffic pattern	North-south dominant	East-west dominant
Loop prevention	STP (blocks links)	L3 routing (ECMP across all links)
Bandwidth utilization	Up to 50% waste (STP blocked ports)	100% via ECMP
Segmentation	4,094 VLANs	16.7M VXLANs
Gateway model	Centralized aggregation switch	Distributed anycast on every leaf
Routing protocol	OSPF / EIGRP	eBGP (RFC 7938)
Overlay protocol	None (VLAN stretching)	VXLAN + EVPN
Scale-out	Redesign required	Add a leaf, connect to all spines

The four core ideas you need to internalize before the rest of this document are:

- **Underlay vs. overlay:** The physical IP network (underlay) carries encapsulated traffic. Virtual networks (overlay) run on top, invisible to the underlay.
- **VXLAN:** Wraps complete L2 Ethernet frames in UDP/IP so they can cross L3 boundaries — stretching virtual segments arbitrarily far.
- **EVPN:** Replaces flood-and-learn with BGP-based MAC and IP advertisement. Every leaf learns remote MACs from control-plane updates, not from data traffic.
- **BGP as underlay:** RFC 7938 explains why BGP outperforms OSPF at scale. The same BGP session carries both IP reachability and EVPN overlay routes via multiple address families.

3 Concept-by-Concept Deep Dive

3.1 Leaf-Spine Topology and the Clos Fabric

A leaf-spine fabric is a **two-tier network topology** derived from Charles Clos's 1953 multistage switching theory. Leaf switches (also called Top-of-Rack or ToR switches) connect directly to servers. Spine switches interconnect all leaf switches in a full mesh. Every leaf connects to every spine. Spines never connect to each other; leaves never connect to each other.

Traffic between any two servers on different leaves traverses exactly **two switch hops** (leaf → spine → leaf). This fixed hop count guarantees predictable, uniform latency. The architecture achieves full **bisectional bandwidth** because ECMP (Equal-Cost Multipath) load-balances flows across all spine paths using a hash of L3/L4 headers. When you need more server ports, add a leaf and connect it to all spines. When you need more inter-leaf bandwidth, add a spine and connect all existing leaves to it. No redesign is required.

In this lab, the topology is minimal but representative: two spines provide two ECMP paths between leaf1 and leaf2. The underlay advertises /32 loopback routes, and each leaf

sees two equal-cost paths to the other leaf's VTEP IP.

3.2 Underlay versus Overlay: Why the Separation Exists

The **underlay** is the physical Layer 3 IP network providing basic IP reachability between all devices. It consists of physical switches running a routing protocol (eBGP in this lab) to establish connectivity between loopback addresses. The underlay is intentionally kept simple: just routed point-to-point links and /32 loopback advertisements.

The **overlay** is the virtual network built on top using tunneling. VXLAN provides the data-plane encapsulation; EVPN provides the control plane. The overlay creates virtualized L2 and L3 segments independent of physical topology. This separation means changes to the underlay (adding links, upgrading devices) do not affect overlay virtual networks, and new virtual networks can be provisioned via software without touching the physical network.

Table 2: Underlay versus overlay: attributes, protocols, and responsibilities

Attribute	**Underlay**	**Overlay**
Purpose	IP transport between VTEPs	Virtual L2/L3 segments for tenants
Protocols	eBGP, BFD, ECMP	VXLAN (data plane), EVPN/MP-BGP (control plane)
Addressing	Point-to-point /31s, loopback /32s	VNIs, tenant IPs, MAC addresses
Complexity	Minimal — just IP routing	All segmentation, policy, multi-tenancy
Visibility	Spine switches see only outer IP headers	Only VTEPs see inner frames

3.3 Why eBGP Replaced OSPF as the DC Underlay

RFC 7938 (*Use of BGP for Routing in Large-Scale Data Centers*, August 2016, by Petr Lapukhov of Facebook/Meta and Arjun Premji of Arista) formalized what hyperscalers had already adopted. BGP is preferred over OSPF/IS-IS for several reasons.

OSPF uses a link-state database flooded to all routers in an area. At scale (thousands of devices), LSDB churn causes massive CPU overhead. **BGP, as a path-vector protocol, naturally limits information propagation scope** — each router only sends its best path to its neighbors, not the entire topology. BGP's AS_PATH attribute provides natural loop prevention in Clos topologies. Each leaf gets a unique ASN; spines share one ASN. The AS_PATH length directly reflects hop count, preventing sub-optimal routing detours.

BGP also supports **multiple address families** over a single session. The same eBGP session carrying IPv4 underlay routes also carries EVPN (L2VPN EVPN) overlay routes — no need for a separate protocol. In this lab, leaf1 (AS 65101) and leaf2 (AS 65102) peer with spines sp1 and sp2 (both AS 65000). The spines naturally re-advertise routes between leaves via standard eBGP mechanics.

BGP unnumbered (RFC 8950, superseding RFC 5549) eliminates the need to assign IPv4 addresses to inter-switch links by using IPv6 link-local addresses with Extended

Next Hop Encoding. This reduces IP address planning to zero per link. FRR supports this natively.

3.4 VXLAN Encapsulation: L2 Frames Riding L3 Packets

VXLAN (Virtual eXtensible LAN, RFC 7348) encapsulates complete Layer 2 Ethernet frames inside UDP/IP packets. The key specifications: **UDP destination port 4789**, a **24-bit VNI** field supporting 16,777,216 segments, and approximately **50 bytes of encapsulation overhead** for IPv4.

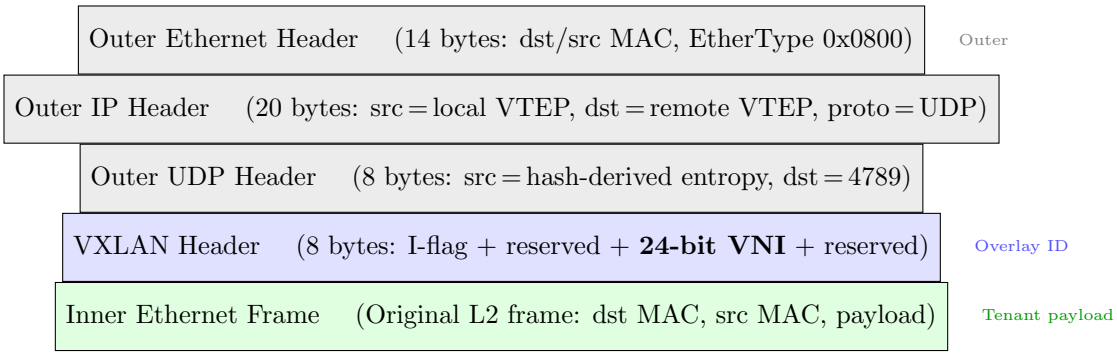

Figure 2: VXLAN encapsulation stack. Underlay switches forward based on the outer IP/UDP headers and have zero awareness of the inner tenant frames or VNI.

The outer UDP source port is derived from a hash of the inner frame's headers, providing **entropy for ECMP load balancing** in the underlay. Underlay switches see only the outer IP/UDP headers and route the packet normally — they have zero awareness of the overlay VNI or inner MAC addresses.

Table 3: VLAN versus VXLAN: a side-by-side comparison

Attribute	**VLAN (802.1Q)**	**VXLAN (RFC 7348)**
Scale	4,094 (12-bit ID)	**16,777,216** (24-bit VNI)
Encapsulation	4-byte tag in Ethernet header	≈50-byte UDP/IP/VXLAN wrapper
L3 boundary	Cannot cross routers	Extends L2 across any IP network
Control plane	STP for loops; flood-and-learn	EVPN (MP-BGP) or multicast flood
Multi-pathing	STP blocks redundant links	Full ECMP via UDP source port entropy
Loop prevention	STP (blocks links)	L3 routing in underlay (no STP)

3.5 VTEP: The Tunnel Endpoint on Every Leaf

A VTEP (VXLAN Tunnel Endpoint) is the device that encapsulates and decapsulates VXLAN traffic. In this lab, leaf1 and leaf2 are VTEPs. The VTEP source IP is **always a loopback address** (leaf1: `10.255.1.1`, leaf2: `10.255.1.2`) because loopbacks never go down, are advertised as /32 host routes in the underlay, and are reachable via any physical path — enabling ECMP.

VTEP-to-VTEP tunnels are **stateless and dynamic**. When EVPN advertises a remote MAC via a Type-2 route, the receiving VTEP learns the remote VTEP IP and can encapsulate traffic directly. There is no tunnel establishment handshake — just encapsulate and send. EVPN Type-3 (Inclusive Multicast) routes advertise VTEP membership in each VNI, enabling dynamic BUM (broadcast/unknown unicast/multicast) flooding trees.

3.6 VNI, L2VNI, and L3VNI: The Two Types of Virtual Network

The **VNI (VXLAN Network Identifier)** is the 24-bit field identifying each virtual segment. The critical distinction is between L2VNI and L3VNI:

Table 4: L2VNI versus L3VNI: purpose, mapping, and behaviour

Attribute	**L2VNI**	**L3VNI**
Purpose	Extends a single L2 broadcast domain (VLAN)	Provides L3 routing transit between subnets in a VRF
Mapping	1:1 with a VLAN/subnet	1:1 with a VRF/tenant
Count per fabric	One per stretched subnet	One per tenant
Bridge SVI	Has an IP address (the default gateway)	**No IP address** — only provides a MAC (the RMAC)
EVPN routes	Type-2 (MAC/IP), Type-3 (IMET)	Type-5 (IP Prefix), Type-2 with L3VNI label
Data carried	Intra-subnet bridged traffic	Inter-subnet routed traffic

In this lab, **VNI 10100** is the L2VNI on leaf1 (associated with `br101`, subnet `192.168.101.0/24`), and **VNI 50100** is the L3VNI shared by both leaves (associated with `br50100` and VRF `vrf-ten1`). Leaf2 has its own L2VNI for `br102` (`192.168.102.0/24`). Because the subnets differ per leaf, there is no L2 stretching — all cross-leaf communication uses the L3VNI.

3.7 L2VNI Bridging: Flood-and-Learn versus EVPN

An L2VNI extends a broadcast domain across the fabric. MAC learning can work two ways. **Flood-and-learn** (original RFC 7348 approach) floods unknown unicast to all remote VTEPs via multicast or head-end replication, and remote VTEPs learn source MACs from the inner headers. This is simple but generates significant BUM traffic at scale.

EVPN control-plane learning replaces flooding with BGP. When a host is detected locally, the leaf advertises the MAC (and optionally IP) via a BGP EVPN Type-2 route. Remote VTEPs install the MAC → VTEP mapping proactively as a static FDB entry — no data-plane flooding required for known unicast. This lab uses EVPN learning via `advertise-all-vni`, which tells FRR's Zebra daemon to scan the kernel for VXLAN interfaces and auto-discover VNIs.

3.8 L3VNI and Symmetric IRB: Routing Across Subnets

When hosts in different subnets communicate across the fabric, the **L3VNI** provides the routed transit path. The **symmetric IRB (Integrated Routing and Bridging)** model works as follows: the ingress VTEP bridges the packet from the local L2VNI, performs an L3 lookup in the VRF, encapsulates with the **L3VNI** and sets the inner destination MAC to the **RMAC (Router MAC)** of the egress VTEP. The egress VTEP decapsulates, sees its own RMAC as the inner destination MAC (triggering L3 routing, not bridging), performs a local L3 lookup, and bridges to the destination host's L2VNI.

The alternative — **asymmetric IRB** — requires every VLAN to be configured on every leaf, which does not scale. Symmetric IRB requires only local VLANs plus one shared L3VNI per VRF. This lab uses symmetric IRB exclusively.

RMAC (Router MAC) is the MAC address of the L3VNI bridge (`br50100`). It is advertised via a BGP Extended Community in Type-2 and Type-5 routes. Without the RMAC in the FDB and neighbor table, the kernel cannot construct the inner Ethernet header for routed VXLAN packets, and **L3VNI forwarding silently fails**.

3.9 EVPN: The Control Plane that Replaced Flooding

EVPN (Ethernet VPN, RFC 7432, February 2015) uses BGP to distribute MAC and IP reachability information. Originally designed for MPLS networks as an evolution of VPLS, RFC 8365 (March 2018) extended EVPN to support VXLAN encapsulation. EVPN replaces flood-and-learn with control-plane MAC learning, enables ARP suppression, supports active-active multi-homing, and integrates L2 bridging with L3 routing in a single control plane.

EVPN uses **MP-BGP extensions** with **AFI 25 (L2VPN) and SAFI 70 (EVPN)**. These values are negotiated via BGP capabilities in the OPEN message. A single BGP session carries both underlay IPv4 routes and overlay EVPN routes — the `address-family l2vpn evpn` stanza activates EVPN on existing BGP neighbors.

3.10 EVPN Route Types: Type-2 and Type-5 in Detail

EVPN defines five primary route types. This lab exercises Type-2 and Type-5:

Type-2 (MAC/IP Advertisement Route) carries a MAC address (mandatory), optional IP address, L2VNI label, optional L3VNI label, and Router MAC extended community. It is generated when a VTEP locally learns a MAC address. When the IP field is populated (via ARP/ND snooping), it enables **ARP suppression** — remote VTEPs can proxy-ARP locally without flooding. In this lab, FRR's `advertise-all-vni` triggers Type-2 generation for every locally learned MAC.

Type-5 (IP Prefix Route, RFC 9136) carries an IP prefix (e.g., `192.168.101.0/24`), the L3VNI label, and the Router MAC extended community. It is generated when `advertise ipv4 unicast` is configured under the VRF's `address-family l2vpn evpn`, which exports routes from the VRF BGP RIB as EVPN Type-5. Type-5 enables inter-subnet routing at the prefix level — not every individual host needs a Type-2 route for L3 reachability.

Table 5: EVPN Type-2 (MAC/IP Advertisement) versus Type-5 (IP Prefix) route comparison

Attribute	**Type-2 (MAC/IP)**	**Type-5 (IP Prefix)**
Carries MAC	Yes (mandatory)	No
Carries IP	Optional (host /32)	Yes (prefix, variable length)
Carries L2VNI	Yes	No
Carries L3VNI	Optional (for symmetric IRB)	Yes (always)
Carries RMAC	Yes (when L3VNI present)	Yes
Granularity	Per-host	Per-subnet
Enables	MAC learning, ARP suppression	Inter-subnet routing
FRR trigger	Automatic (`advertise-all-vni`)	`advertise ipv4 unicast`

Type-3 (Inclusive Multicast Ethernet Tag) advertises VTEP membership in a VNI and establishes the BUM flooding tree. **Type-1 (Ethernet Auto-discovery)** and **Type-4 (Ethernet Segment)** support multi-homing, which is not used in this lab.

3.11 Route Distinguisher and Route Target

Route Distinguisher (RD) is an 8-byte value prepended to EVPN NLRI to make routes globally unique in BGP. Without RDs, two tenants using the same MAC address would create ambiguous routes. FRR auto-derives RD as `RouterID:VNI` (Type 1 format). RD carries no semantic meaning — it is purely a uniqueness mechanism.

Route Target (RT) is a BGP Extended Community that controls import/export of routes between VRFs. When a VTEP advertises an EVPN route, it attaches an Export RT. Receiving VTEPs compare the Export RT against their Import RTs — if they match, the route is imported into that VRF/VNI. **For route exchange, the export RT on the originating leaf must match the import RT on the receiving leaf.**

In this lab, the RT values are non-standard: leaf1 exports `65101:50100` and imports `65102:50100`, while leaf2 does the reverse. This uses the per-leaf ASN as the administrator field. The standard approach would use a fixed value like `1:50100` across all leaves, simplifying configuration and scaling.

3.12 Anycast Gateway: Every Leaf Is the Default Gateway

All leaves share the **same gateway IP and MAC** for each subnet. In this lab, leaf1's `br101` has IP `192.168.101.1/24` and leaf2's `br102` has IP `192.168.102.1/24`. Because each leaf has its own unique subnet (no L2 stretching), the anycast concept applies mainly if the design were extended with stretched L2VNIs. The `advertise-default-gw` command in FRR advertises the gateway IP/MAC via Type-2 routes. `advertise-svi-ip` advertises the SVI's real IP address.

3.13 VRF: Isolated Routing Tables for Multi-Tenancy

A VRF (Virtual Routing and Forwarding) instance provides an isolated routing table. Linux implements VRFs as network devices with associated routing tables:

Listing 1: Linux VRF creation and interface enslavement

```
ip link add vrf-ten1 type vrf table 10
ip link set vrf-ten1 up
ip link set br50100 master vrf-ten1   # Enslave L3VNI bridge to VRF
ip link set br101   master vrf-ten1   # Enslave L2VNI bridge to VRF
```

Routes for the VRF go into **table 10**. The kernel's `l3mdev-table` ip rule (priority 1000, auto-created) directs packets on VRF-enslaved interfaces to the correct table. Each tenant gets their own VRF mapped to a unique L3VNI. In FRR, the mapping is configured as `vni 50100` under `vrf vrf-ten1`.

3.14 ARP Suppression: Eliminating Broadcast Floods

When EVPN Type-2 routes carry both MAC and IP, each VTEP builds a proxy-ARP table. When a local host ARPs for a remote host, the VTEP answers directly from its cache instead of flooding the ARP request across the fabric. This converts ARP from broadcast to unicast, dramatically reducing BUM traffic in large deployments. It is enabled with:

```
bridge link set dev vxlanXXX neigh_suppress on
```

3.15 BFD: Sub-Second Failure Detection

BFD (Bidirectional Forwarding Detection, RFC 5880) provides **sub-second failure detection** between adjacent forwarding engines, independent of routing protocols. It sends lightweight UDP packets (port 3784) at configurable intervals. Typical configuration: **300 ms interval × 3 multiplier = 900 ms detection time**. When BFD detects a failure, it immediately notifies BGP, which tears down the session without waiting for the hold timer (default 180 s, or 9 s with datacenter profile). In this lab, `bfdd=yes` in the daemons file enables BFD, and `neighbor X bfd` in the FRR config attaches BFD to each BGP session.

3.16 Linux Bridge, FDB, and Neighbor Table

A **Linux bridge** is a kernel module that behaves like a switch, forwarding frames between ports based on MAC address learning. In EVPN/VXLAN, each VNI has a VXLAN interface enslaved to a bridge. The bridge connects local host-facing ports with the VXLAN tunnel interface.

The **FDB (Forwarding Database)** maps MAC addresses to either local ports or remote VTEP IPs. For EVPN-learned MACs, entries appear with the `extern_learn` flag (kernel will not garbage-collect them) and the `self` flag (programmed on the VXLAN device with a `dst` field containing the remote VTEP IP). The all-zeros MAC (`00:00:00:00:00:00`) with a `dst` IP is the BUM flooding entry for that remote VTEP.

The **neighbor table** (`ip neigh`) maps IP addresses to MAC addresses. For L3VNI forwarding, the RMAC of each remote VTEP must appear as a neighbor entry on the L3VNI bridge. If the entry is in **FAILED state** (resolution failed), the kernel cannot construct the inner Ethernet header and **silently drops packets** — this is the most common cause of L3VNI forwarding failures.

3.17 FRRouting Architecture: Zebra and the Daemon Model

FRRouting is an open-source routing suite for Linux, forked from Quagga in April 2017 by Cumulus Networks, 6WIND, and others. It is used by NVIDIA Cumulus Linux, Microsoft SONiC, DENT, and VyOS. FRR uses a modular architecture: each protocol runs as an independent daemon (bgpd, bfdd, ospfd, etc.), communicating with the **Zebra daemon** via the ZAPI (Zebra API) over Unix sockets.

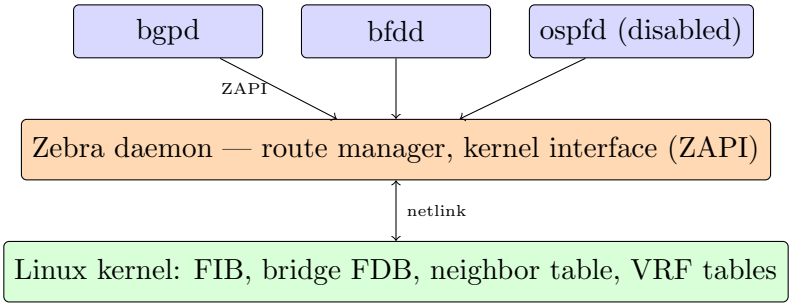

Figure 3: FRR daemon architecture. Zebra is the central manager: it receives routes from protocol daemons, selects the best route, and installs it into the Linux kernel via netlink.

Zebra is the central routing manager. It receives routes from protocol daemons, performs route selection across all sources using administrative distance, and installs the best routes into the Linux kernel via netlink. Critical distinction: **a route visible in FRR (`show ip route`) may not be in the Linux kernel** if Zebra has not installed it or if installation failed. Always verify with both `vtysh -c "show ip route"` and `ip route show table N`.

For EVPN specifically, Zebra discovers VXLAN interfaces from the kernel (triggered by `advertise-all-vni`), sends VNI_ADD messages to bgpd, and programs FDB entries, neighbor entries, and routes into the kernel based on EVPN routes received by bgpd.

3.18 Kernel Routing Tables and rp_filter

Linux supports multiple routing tables. **Table 254 (main)** is the default; **table 255 (local)** holds local/broadcast addresses; custom tables (like **table 10** for `vrf-ten1`) isolate VRF routing. The `ip rule` system directs lookups: the `l3mdev-table` rule (priority 1000) automatically steers VRF traffic to the correct table.

rp_filter (Reverse Path Filtering) is a kernel security feature that drops packets whose source IP would not be routed back out the arriving interface. It **must be disabled (set to 0)** for EVPN/VXLAN because VXLAN-encapsulated packets arrive on physical interfaces with source IPs (remote VTEP loopbacks) that may be reachable only via a different path. Critical: Linux takes the **MAX** of `conf/all/rp_filter` and `conf/{interface}/rp_filter`, so both must be set to 0.

3.19 Containerlab: Networking Labs in Containers

Containerlab is a CLI tool for creating networking labs using containers, developed by Nokia/SR Labs. It reads a YAML topology file, starts Docker containers for each node,

and creates **veth pairs** between containers to simulate physical links (bypassing Docker networking for the data plane). Key features used in this lab:

- `binds`: Mount host files into containers (e.g., FRR configs → `/etc/frr/frr.conf`)
- `exec`: Run commands at startup (e.g., create bridges, set sysctls)
- `sysctls`: Set kernel parameters like `ip_forward=1` and `rp_filter=0`

Deploy with `containerlab deploy -t vxlan-static.clab.yml`; destroy with `containerlab destroy -t vxlan-static.clab.yml`.

4 Repository File Walkthrough

4.1 `vxlan-static.clab.yml`: The 6-Node Topology

This is the main Containerlab topology file defining all nodes and links. It creates:

- **sp1, sp2**: Spine switches using `frrouting/frr:latest`
- **leaf1, leaf2**: Leaf/VTEP switches using `frrouting/frr:latest`, with sysctls setting `net.ipv4.ip_forward=1` and `net.ipv4.conf.all.rp_filter=0`
- **h1, h2**: Hosts using `alpine:3.20`

Each leaf binds `configs/daemons`, its `frr.conf`, `vtysh.conf`, and `init.sh` into the container. The `exec` directive runs `init.sh` at startup. Links connect leaf1↔sp1, leaf1↔sp2, leaf2↔sp1, leaf2↔sp2, leaf1↔h1, leaf2↔h2.

Listing 2: Key sections of `vxlan-static.clab.yml` showing binds and exec for a leaf node

```
leaf1:
  binds:
    - configs/daemons:/etc/frr/daemons
    - configs/vtysh.conf:/etc/frr/vtysh.conf
    - configs/leaf1/frr.conf:/etc/frr/frr.conf
    - configs/leaf1/init.sh:/init.sh
  exec:
    - sh -c "chmod +x /init.sh && /init.sh"
```

4.2 `configs/daemons`: Which FRR Processes Run

Listing 3: `configs/daemons`: FRR daemon enable/disable flags

```
zebra=yes    # Route manager, kernel interface
bgpd=yes     # BGP for underlay + EVPN overlay
bfdd=yes     # BFD for fast failure detection
ospfd=no     # Not used -- eBGP is the underlay protocol
ospf6d=no
ripd=no
ripngd=no
isisd=no
pimd=no
ldpd=no
nhrpd=no
```

```
eigrpd=no
babeld=no
sharpd=no
pbrd=no
fabricd=no
vrrpd=no
```

This file is bound to `/etc/frr/daemons` in every FRR container. The explicit `ospfd=no` confirms this is a pure eBGP fabric. This file is **shared across all nodes** — both spines and leaves load it, even though BFD is used primarily for leaf-spine links.

4.3 `configs/leaf1/frr.conf`: The Leaf BGP + EVPN Configuration

Listing 4: Full `configs/leaf1/frr.conf`

```
frr defaults traditional
hostname leaf1
service integrated-vtysh-config
!
interface lo
 ip address 10.255.1.1/32
!
interface eth1
 ip address 10.0.0.1/31
!
interface eth2
 ip address 10.0.0.5/31
!
vrf vrf-ten1
 vni 50100
exit-vrf
!
router bgp 65101
 bgp router-id 10.255.1.1
 no bgp ebgp-requires-policy
 neighbor 10.0.0.0 remote-as 65000
 neighbor 10.0.0.0 bfd
 neighbor 10.0.0.4 remote-as 65000
 neighbor 10.0.0.4 bfd
 !
 address-family ipv4 unicast
  neighbor 10.0.0.0 activate
  neighbor 10.0.0.4 activate
  network 10.255.1.1/32
 exit-address-family
 !
 address-family l2vpn evpn
  neighbor 10.0.0.0 activate
  neighbor 10.0.0.4 activate
  advertise-all-vni
  advertise-default-gw
  advertise-svi-ip
 exit-address-family
!
router bgp 65101 vrf vrf-ten1
```

```
 !
 address-family ipv4 unicast
  redistribute connected
 exit-address-family
 !
 address-family l2vpn evpn
  advertise ipv4 unicast
  route-target import 65102:50100
 exit-address-family
!
line vty
```

Leaf1 is configured as AS 65101 with router-id `10.255.1.1` (its loopback/VTEP IP). It peers with spines on `eth1` and `eth2` as `remote-as 65000`. Under `address-family l2vpn evpn`, it activates EVPN on both spine neighbors and enables:

- `advertise-all-vni`: Triggers Zebra to discover VNIs from kernel VXLAN interfaces
- `advertise-default-gw`: Advertises gateway IP/MAC via Type-2 routes
- `advertise-svi-ip`: Advertises the SVI's operational IP address

A separate BGP instance for VRF `vrf-ten1` maps `vni 50100` (the L3VNI), redistributes connected routes, and `advertise ipv4 unicast` under `l2vpn evpn` generates Type-5 routes. Route targets: **export** `65101:50100`**, import** `65102:50100` — importing only from leaf2's ASN-based RT.

4.4 `configs/sp1/frr.conf` and `configs/sp2/frr.conf`: Spine as EVPN Relay

Listing 5: Full `configs/sp1/frr.conf`

```
frr defaults traditional
hostname sp1
service integrated-vtysh-config
!
interface lo
 ip address 10.255.0.1/32
!
interface eth1
 ip address 10.0.0.0/31
!
interface eth2
 ip address 10.0.0.2/31
!
router bgp 65000
 bgp router-id 10.255.0.1
 no bgp ebgp-requires-policy
 neighbor 10.0.0.1 remote-as 65101
 neighbor 10.0.0.1 bfd
 neighbor 10.0.0.3 remote-as 65102
 neighbor 10.0.0.3 bfd
 !
 address-family ipv4 unicast
  neighbor 10.0.0.1 activate
  neighbor 10.0.0.3 activate
```

```
  network 10.255.0.1/32
 exit-address-family
 !
 address-family l2vpn evpn
  neighbor 10.0.0.1 activate
  neighbor 10.0.0.3 activate
 exit-address-family
!
line vty
```

Spines are in AS 65000, peering with both leaves. They activate `l2vpn evpn` address-family but do **NOT** have `advertise-all-vni` — this is correct because spines do not originate EVPN routes. They purely relay EVPN updates between leaves via standard eBGP re-advertisement. This is a potential confusion point: someone might think spines need `advertise-all-vni`, but they do not because they have no VXLAN interfaces.

4.5 `configs/leaf1/init.sh`: Linux Dataplane Setup (With Dead Code Bug)

Listing 6: Active (reachable) portion of `configs/leaf1/init.sh`

```
#!/bin/sh
set -e

# Tenant VRF
ip link add vrf-ten1 type vrf table 10 2>/dev/null || true
ip link set vrf-ten1 up

# Tenant L2 segment (VNI 10100) - br101 + access port eth3
ip link add br101 type bridge 2>/dev/null || true
ip link set br101 up
ip link set dev br101 address 02:00:00:00:01:01
ip addr replace 192.168.101.1/24 dev br101
ip link set br101 master vrf-ten1 || true

ip link set eth3 up
ip link set eth3 master br101

# L2VNI VXLAN interface
ip link add vxlan101 type vxlan id 10100 dstport 4789 local 10.255.1.1
    2>/dev/null || true
ip link set vxlan101 up
ip link set vxlan101 master br101

# L3VNI (VNI 50100) - br50100 has no IP
ip link add vxlan50100 type vxlan id 50100 dstport 4789 local
   10.255.1.1 2>/dev/null || true
ip link set vxlan50100 up
ip link add br50100 type bridge 2>/dev/null || true
ip link set br50100 up
ip link set vxlan50100 master br50100
ip link set br50100 master vrf-ten1 || true

# Disable rp_filter
sysctl -w net.ipv4.conf.all.rp_filter=0     >/dev/null
sysctl -w net.ipv4.conf.default.rp_filter=0  >/dev/null
```

```
sysctl -w net.ipv4.conf.vxlan50100.rp_filter=0 >/dev/null 2>&1 || true
sysctl -w net.ipv4.conf.br50100.rp_filter=0   >/dev/null 2>&1 || true
sysctl -w net.ipv4.conf.br101.rp_filter=0     >/dev/null 2>&1 || true

exit 0
# ===== EVERYTHING BELOW IS DEAD CODE (never reached) =====
```

Critical bug: The `exit 0` statement on its own line terminates script execution. All subsequent code — including multiple RMAC watchdog loops, static route commands, and repeated bridge setup blocks — is *completely unreachable*. This is the most significant defect in the repository. See Section 8 for full analysis.

4.6 Host Init Scripts: Simple IP Assignment

Listing 7: `configs/h1/init.sh`

```
#!/bin/sh
set -e
ip addr flush dev eth1 || true
ip addr add 192.168.101.11/24 dev eth1
ip link set eth1 up
ip route replace default via 192.168.101.1
```

Listing 8: `configs/h2/init.sh`

```
#!/bin/sh
set -e
ip addr flush dev eth1 || true
ip addr add 192.168.102.12/24 dev eth1
ip link set eth1 up
ip route replace default via 192.168.102.1
```

These scripts are clean, minimal, and correct. h1 assigns `192.168.101.11/24` to `eth1` and sets the default route via `192.168.101.1` (leaf1's `br101`). h2 assigns `192.168.102.12/24` and routes via `192.168.102.1` (leaf2's `br102`).

4.7 `configs/vtysh.conf`: Empty File

The file `configs/vtysh.conf` is bound to `/etc/frr/vtysh.conf` in every node but contains **no content**. This provides no configuration value; the bind could be removed entirely or the file should contain at least `service integrated-vtysh-config` to enable single-file FRR config mode.

4.8 Generated Artifacts: `clab-underlay/` and `clab-vxlan-static/`

The `clab-underlay/` and `clab-vxlan-static/` directories contain Containerlab-generated state artifacts (`ansible-inventory.yml`, `nornir-simple-inventory.yml`, `topology-data.json`). These are regenerated on every `clab deploy` and should be listed in `.gitignore`. Committing them to the repository adds noise without value.

4.9 `underlay.clab.yml`: Simpler Pre-EVPN Topology

This file defines the same four-node underlay (sp1, sp2, leaf1, leaf2) without hosts or init scripts. It was used for underlay-only testing before EVPN was added. It serves as a useful intermediate step for learning: deploy the underlay, validate eBGP sessions and ECMP, then add the EVPN overlay on top.

5 End-to-End Packet Walkthroughs

5.1 h1 Pings h2: Inter-Subnet Routing via Symmetric IRB

Step 1 — h1 ARPs for its gateway. h1 (`192.168.101.11`) wants to reach h2 (`192.168.102.12`). Different subnet, so h1 ARPs for gateway `192.168.101.1`. leaf1's `br101` responds with its bridge MAC. h1 sends: `Src MAC = h1-MAC, Dst MAC = br101-MAC, Src IP = 192.168.101.11, Dst IP = 192.168.102.12`.

Step 2 — leaf1 performs L3 routing in VRF. leaf1 sees `Dst MAC = its own br101 MAC` ⇒ packet must be routed. L3 lookup in `vrf-ten1`'s routing table (table 10) finds `192.168.102.0/24` via next-hop `10.255.1.2` (leaf2's VTEP) with L3VNI 50100 and RMAC = leaf2's `br50100` MAC.

Step 3 — VXLAN encapsulation with L3VNI. leaf1 constructs the following layered packet:

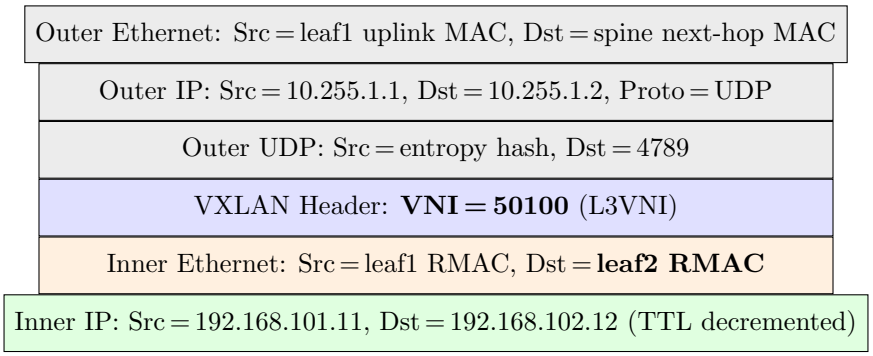

Figure 4: h1→h2 encapsulated packet at leaf1 egress. The L3VNI (50100) signals egress leaf to route; the inner Dst MAC is the remote RMAC, not h2's MAC.

Step 4 — Underlay transit. The packet is routed as normal IP through sp1 or sp2 (ECMP). Spines see only the outer IP headers and forward based on the destination `10.255.1.2/32` route learned from eBGP.

Step 5 — leaf2 decapsulates. leaf2 receives UDP:4789, VNI = 50100 → maps to `vrf-ten1`. Inner Dst MAC = leaf2's own RMAC → triggers **L3 routing** (not bridging).

Step 6 — leaf2 routes to local host. L3 lookup in `vrf-ten1` for `192.168.102.12` → directly connected on `br102`. leaf2 resolves h2's MAC via neighbor table, sends: `Src MAC = br102-MAC, Dst MAC = h2-MAC`. Frame exits `eth3` to h2.

The symmetric nature means both directions use the same L3VNI (50100). Return traffic (h2→h1) follows the same pattern with leaf2 as ingress and leaf1 as egress.

5.2 Type-2 Route Learning Flow

When h1 sends its first frame, leaf1's `br101` learns h1's MAC via standard bridge learning. Zebra receives a `RTM_NEWNEIGH` netlink notification from the kernel, notifies bgpd, which generates a Type-2 route:

Listing 9: EVPN Type-2 NLRI structure generated for h1

```
[2]:[0]:[48]:[h1-MAC]:[32]:[192.168.101.11]
  RD: auto-derived (10.255.1.1:10100)
  RT export: 65101:10100 (L2VNI)
  RT export: 65101:50100 (L3VNI)
  VNI labels: L2=10100, L3=50100
  RMAC: leaf1's br50100 MAC
  Next-hop: 10.255.1.1
```

This route flows via eBGP through sp1 and sp2 to leaf2. leaf2 matches the import RT, and Zebra programs:

- An FDB entry: `h1-MAC dst 10.255.1.1` on `vxlan101`
- A neighbor entry: `192.168.101.11 lladdr h1-MAC` on `br50100`

5.3 Type-5 Route Learning Flow

leaf1's VRF BGP has `redistribute connected` injecting `192.168.101.0/24` into the VRF RIB. `advertise ipv4 unicast` exports this as Type-5:

Listing 10: EVPN Type-5 NLRI structure for leaf1's connected subnet

```
[5]:[0]:[24]:[192.168.101.0]
  RD: auto-derived
  RT export: 65101:50100
  L3VNI: 50100
  RMAC: leaf1's br50100 MAC
  Next-hop: 10.255.1.1
```

leaf2 receives this, matches import RT `65101:50100`, and installs in kernel:

Listing 11: Route installed by Zebra in leaf2 kernel table 10

```
ip route show table 10
# 192.168.101.0/24 via 10.255.1.1 dev vxlan50100 proto bgp metric 20
   onlink
```

5.4 BFD and Failure Convergence

When leaf1's `eth1` (toward sp1) goes down, BFD on both leaf1 and sp1 detect the failure within the configured timer ($\approx$300 ms $\times$ 3 = 900 ms). BFD immediately signals BGP, which tears down the session — bypassing the normal hold timer wait. BGP withdraws routes learned from sp1. If `eth2` (toward sp2) is still up, ECMP simply removes the sp1 path and continues forwarding via sp2 only. If both uplinks fail, all routes are withdrawn and h1 loses connectivity until links recover and BGP reconverges.

Convergence measurement approach:

Listing 12: Failure injection and convergence measurement

```
# Probe traffic at 0.2s intervals
sudo docker exec -it clab-vxlan-static-h1 ping -i 0.2 -c 125
    192.168.102.12 &

# Inject failure
date +"DOWN start  %H:%M:%S.%3N"
sudo docker exec -it clab-vxlan-static-leaf1 ip link set eth1 down
sudo docker exec -it clab-vxlan-static-leaf1 ip link set eth2 down
date +"DOWN done   %H:%M:%S.%3N"
sleep 2
sudo docker exec -it clab-vxlan-static-leaf1 ip link set eth1 up
sudo docker exec -it clab-vxlan-static-leaf1 ip link set eth2 up

# Loss window = lost_packets x probe_interval (0.2s each)
```

6 Control Plane versus Data Plane

The **control plane** consists of all protocols that exchange reachability information: eBGP sessions carrying IPv4 underlay routes and EVPN overlay routes, BFD sessions monitoring link health, and Zebra's interaction with the kernel via netlink. The control plane tells switches *where* things are but does not forward user traffic.

The **data plane** is the actual packet forwarding path: Linux bridges performing MAC lookups, the VXLAN kernel module encapsulating/decapsulating frames, the kernel's FIB performing route lookups in VRF tables, and the neighbor table resolving next-hop MACs. Data plane operations happen in the kernel at line rate (for software); control plane operations happen in userspace daemons.

Table 6: Control plane (FRR) versus data plane (Linux kernel): precise separation of responsibilities

Attribute	**Control Plane (FRR)**	**Data Plane (Linux Kernel)**
Components	bgpd, zebra, bfdd	bridge module, VXLAN module, FIB, neighbor table
Runs in	Userspace daemons	Kernel space
Protocol	BGP, BFD, ZAPI	Ethernet bridging, IP routing, VXLAN encapsulation
Verification	`show bgp`, `show evpn`	`bridge fdb`, `ip route`, `ip neigh`
State	BGP RIB, EVPN VNI table	FDB, FIB, neighbor cache
Failure mode	Routes learned but not installed	Packets dropped silently, no error message

A critical debugging pattern: FRR shows the route exists (`show bgp l2vpn evpn route`) but the kernel does not have it (`ip route show table 10` is empty). This means Zebra failed to install the route — check Zebra logs and interface state.

The flow of information is strictly unidirectional in normal operation:

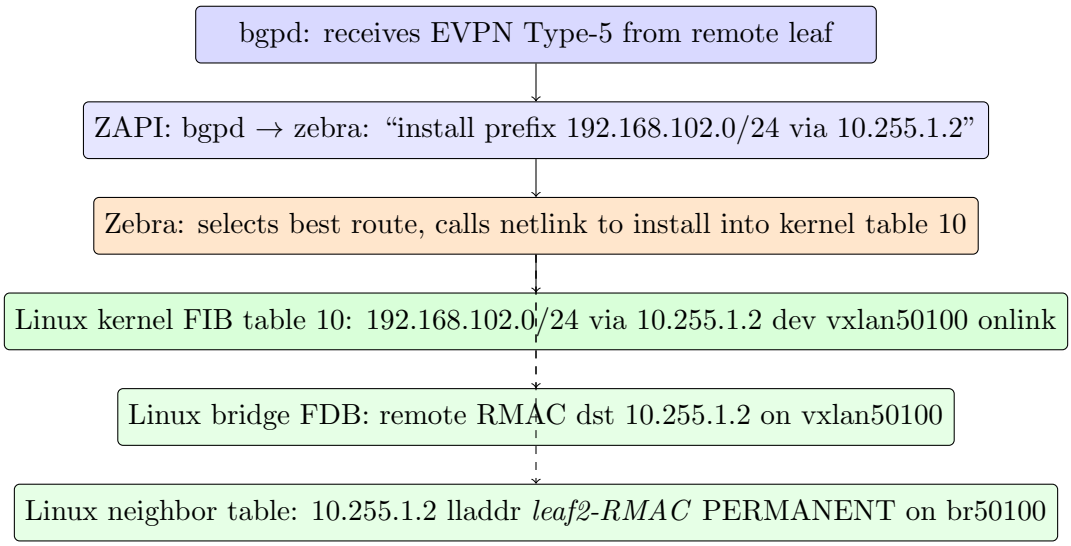

Figure 5: Information flow from EVPN BGP update through FRR Zebra into the Linux kernel. Dashed arrows indicate that FDB and neighbor programming may require the dead-code workaround paths to be fixed first in this lab.

Why Route Presence in FRR Is Not Enough

If the `ip neigh` entry for the remote VTEP IP (`10.255.1.2`) is in FAILED state on `br50100`, the kernel cannot resolve the RMAC for the inner Ethernet header. This means that even if `ip route show table 10` shows the correct next-hop, and even if `show bgp l2vpn evpn route` shows the Type-5 route, **packets are silently dropped** at the point of inner-frame construction. This is why the dead RMAC watchdog code in `init.sh` was written in the first place — to work around cases where Zebra's automatic neighbor programming was unreliable in this containerized environment.

7 Verification and Troubleshooting Lab Guide

7.1 FRR Verification Commands

Check Underlay BGP Sessions

Listing 13: Check BGP session state on all nodes

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show ip bgp
    summary"
sudo docker exec -it clab-vxlan-static-leaf2 vtysh -c "show ip bgp
    summary"
sudo docker exec -it clab-vxlan-static-sp1   vtysh -c "show ip bgp
    summary"
sudo docker exec -it clab-vxlan-static-sp2   vtysh -c "show ip bgp
    summary"
```

Good output (leaf1): Two neighbors in `Established` state with non-zero `PfxRcd` (prefixes received) and non-zero uptime.

Listing 14: Expected healthy BGP summary on leaf1

```
Neighbor    V   AS      Up/Down    State/PfxRcd PfxSnt
10.0.0.0    4   65000   04:10:22   2            1
10.0.0.4    4   65000   04:10:20   2            1
```

Broken output: State shows `Active` or `Connect` instead of a prefix count. Check interface state, AS numbers, and that BFD is not flapping.

Check EVPN Session Status

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show bgp l2vpn
    evpn summary"
```

`PfxRcd` should be >0 (receiving EVPN routes from remote leaf via spines).

View EVPN Type-2 and Type-5 Routes

Listing 15: Inspect EVPN route tables

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show bgp l2vpn
    evpn route type 2"
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show bgp l2vpn
    evpn route type 5"
```

Good: Type-2 entries show remote MACs with correct VNI labels and RMAC. Type-5 entries show `192.168.102.0/24` with L3VNI 50100 and RMAC pointing to leaf2's `br50100` MAC.

Check VNI Status

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show evpn vni"
```

Good:

```
VNI     Type  VxLAN IF     #MACs  #ARPs  #Remote VTEPs  Tenant VRF
10100   L2    vxlan101     2      2      1              vrf-ten1
50100   L3    vxlan50100   1      1      n/a            vrf-ten1
```

Both VNIs should show the correct `Tenant VRF` and non-zero MAC/ARP counts.

VRF Routing Table in FRR

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show bgp vrf
    vrf-ten1 ipv4 unicast"
```

Good:

```
*> 192.168.101.0/24   0.0.0.0            0       32768 ?    # Local
    connected
*> 192.168.102.0/24   10.255.1.2                 0 65000 65102 ?   # From
     leaf2
```

BFD Peer Status

```
sudo docker exec -it clab-vxlan-static-leaf1 vtysh -c "show bfd peers"
```

Good: All peers in `up` state with recent `last-failure` timestamps.

7.2 Linux Kernel Verification Commands

FDB Entries on L3VNI

Listing 16: Inspect VXLAN FDB for L3VNI device

```
sudo docker exec -it clab-vxlan-static-leaf1 bridge fdb show dev
    vxlan50100 | head -n 30
```

Good:

```
aa:bb:cc:00:00:02 dst 10.255.1.2 self static extern_learn
00:00:00:00:00:00 dst 10.255.1.2 self permanent
```

The first entry maps leaf2's RMAC to leaf2's VTEP IP (required for L3VNI routing). The second is the BUM flood entry. **Missing RMAC entry = L3VNI routing broken.**

Neighbor Table on L3VNI Bridge

```
sudo docker exec -it clab-vxlan-static-leaf1 ip neigh show dev br50100
```

Good:

```
10.255.1.2 lladdr aa:bb:cc:00:00:02 PERMANENT
```

Broken:

```
10.255.1.2  FAILED
```

A `FAILED` state means ARP resolution failed. The kernel cannot construct the inner Ethernet header. Inter-subnet traffic silently drops. Recover by:

Listing 17: Manual RMAC repair after FAILED neighbor state

```
ip neigh replace 10.255.1.2 lladdr <leaf2-RMAC> dev br50100 nud
    permanent
bridge fdb replace <leaf2-RMAC> dev vxlan50100 dst 10.255.1.2 self
    permanent
```

VRF Routing Table in Kernel

Listing 18: Verify VRF routes are installed in kernel table 10

```
sudo docker exec -it clab-vxlan-static-leaf1 ip route show table 10
```

Good:

```
192.168.101.0/24 dev br101 proto kernel scope link src 192.168.101.1
192.168.102.0/24 via 10.255.1.2 dev vxlan50100 proto bgp metric 20
    onlink
```

Both the local connected route and the EVPN-learned remote route must be present.

Policy Routing Rules

```
sudo docker exec -it clab-vxlan-static-leaf1 ip rule show
```

Good:

```
0:      from all lookup local
1000:   from all lookup [l3mdev-table]
32766:  from all lookup main
32767:  from all lookup default
```

The `l3mdev-table` rule at priority 1000 must exist for VRF routing to function.

Flush Route Cache After Changes

```
sudo docker exec -it clab-vxlan-static-leaf1 ip route flush cache
```

Always flush the route cache after making changes or injecting failures to avoid stale forwarding decisions.

7.3 Host-Level Reachability Tests

Listing 19: Full host-to-host and gateway reachability tests

```
# h1 to local gateway
sudo docker exec -it clab-vxlan-static-h1 ping -c 2 192.168.101.1

# h1 to h2 (inter-subnet, tests entire EVPN stack)
sudo docker exec -it clab-vxlan-static-h1 ping -c 3 192.168.102.12

# h2 to local gateway
sudo docker exec -it clab-vxlan-static-h2 ping -c 2 192.168.102.1

# Loopback-to-loopback (must specify source interface)
sudo docker exec -it clab-vxlan-static-leaf1 ping -c 3 -I 10.255.1.1
    10.255.1.2
```

Note: Loopback-to-loopback pings *require* specifying the source with `-I` because the default source selection may pick the management interface (`eth0`), which is not in the underlay routing table.

7.4 Common Failure Patterns and Remediation

Table 7: Common EVPN/VXLAN failure symptoms, root causes, and remediation steps

Symptom	Root Cause	Remediation
EVPN routes in FRR but not in kernel	VNI/VRF mapping missing; bridge not enslaved to VRF	Verify `vni 50100` in VRF config; check `bridge link show`; restart FRR
`ip neigh FAILED` on br50100	RMAC not resolved; br50100 not connected to underlay	Manually program `ip neigh replace ... nud permanent`; check FDB
Ping works one way only	rp_filter dropping asymmetric VXLAN; RT mismatch on return path	Set `rp_filter=0`; verify RT export/import symmetry
FDB entry missing for remote VTEP	Type-3 IMET not received; VNI not discovered	Check `show evpn vni`; verify `advertise-all-vni`; check VXLAN interface state
BGP sessions not forming	AS number mismatch; BFD flapping; interface down	Check `show bgp summary`; verify AS numbers in `frr.conf`; check `show bfd peers`

8 Repository Weaknesses and Cleanup Opportunities

8.1 Critical Bug: Dead Code After `exit 0`

The most significant defect appears in both `leaf1/init.sh` and `leaf2/init.sh`. The `exit 0` statement terminates script execution, making all subsequent code **completely unreachable**. This includes:

- RMAC watchdog loops (duplicated 3–4 times within the same script), each using slightly different RMAC extraction logic (awk-based, sed-based, etc.)
- Static route commands: `ip route replace 192.168.102.0/24 via 10.255.1.2 dev br50100 table 10 onlink`
- Repeated bridge setup blocks (`ip link add br50100`, `ip link set br50100 up`, etc.)

The fix is straightforward: either remove the dead code or move `exit 0` to the actual end of the file. The RMAC watchdog logic, even if reachable, is redundant — FRR's Zebra daemon handles RMAC programming automatically via EVPN when configured correctly.

Listing 20: Dead code pattern in init.sh – everything after exit 0 never runs

```
sysctl -w net.ipv4.conf.br101.rp_filter=0 >/dev/null 2>&1 || true

exit 0       # <-- Script terminates here

# UNREACHABLE: RMAC watchdog (attempt 1 of 4)
for i in $(seq 1 30); do
  RM2=$(vtysh -c "show bgp l2vpn evpn route type 5" 2>/dev/null | awk
      '/Rmac/{print $NF}' | tail -n 1)
  echo "$RM2" | grep -Eq '([0-9a-f]{2}:){5}[0-9a-f]{2}' && break
```

```
  sleep 0.2
done
# ... duplicated 3 more times with 50/60 loop iterations instead of 30
```

8.2 Empty `vtysh.conf`

The file `configs/vtysh.conf` is bound to `/etc/frr/vtysh.conf` in every node container but contains **no content**. This provides no configuration value. Either add meaningful content (e.g., `service integrated-vtysh-config`) or remove the bind from `vxlan-static.clab.yml` to reduce noise.

8.3 Generated Artifacts Committed to Repository

The `clab-underlay/` and `clab-vxlan-static/` directories contain Containerlab-generated state artifacts:

- `ansible-inventory.yml`
- `nornir-simple-inventory.yml`
- `topology-data.json`

These are regenerated on every `clab deploy` with management IP addresses that differ between deployments. Committing them to the repository adds noise and will conflict for other users. They should be added to `.gitignore`:

Listing 21: Recommended .gitignore additions

```
clab-*/
*.clab.yml.annotations.json
```

8.4 Non-Standard Route Target Values

Route targets use per-leaf ASN prefixes (`65101:50100`/`65102:50100`) instead of a uniform value like `1:50100`. This works for a two-leaf lab but scales poorly — adding a third leaf (AS 65103) would require updating import RTs on all existing leaves to add `65103:50100`. Standard practice uses a fixed administrator value (e.g., `1:50100`) across all leaves so no per-leaf RT changes are needed when the fabric grows.

8.5 L2VNI Relies Entirely on `advertise-all-vni`

The L2VNI (`vxlan101`, VNI 10100) has no explicit FRR configuration stanza. It works because `advertise-all-vni` auto-discovers VXLAN interfaces from the kernel. This is valid and common in FRR/Cumulus deployments, but the implicit dependency is worth documenting explicitly for maintainability.

8.6 No Cross-Leaf L2 Stretching

leaf2 creates no `vxlan101` (or VNI 10100 equivalent). Each leaf has its own unique L2VNI. All cross-leaf communication uses L3VNI routing only. This is a deliberate design choice (asymmetric subnets) but means the lab does not demonstrate L2VNI stretching — a

common real-world use case. A future enhancement would add a shared L2VNI (e.g., VNI 20100 for `192.168.200.0/24`) accessible from both leaves.

8.7 Summary of Issues

Table 8: Repository defects and cleanup opportunities with severity ratings

Issue	**Severity**	**Recommended Fix**
Dead code after `exit 0` (both leaf init scripts)	Critical	Move `exit 0` to end or remove dead code
Empty `vtysh.conf`	Low	Add content or remove bind
Generated artifacts in repo	Medium	Add to `.gitignore`
Non-standard Route Target format	Medium	Use `1:50100` uniformly
No `.gitignore` for clab outputs	Medium	Add `clab-*/` to `.gitignore`
No L2VNI stretching demo	Enhancement	Add shared L2VNI across both leaves

9 Interview Preparation

9.1 The 30-Second Pitch

"I built a full EVPN/VXLAN data center fabric lab using FRRouting and Containerlab. It is a leaf-spine topology with two spines and two leaves running eBGP as the underlay, VXLAN for Layer 2 over Layer 3 encapsulation, and EVPN as the BGP-based control plane. Hosts on different subnets communicate through symmetric IRB routing using a shared L3VNI. I configured the complete Linux dataplane — bridges, VXLAN interfaces, VRFs, and FDB entries — and the FRR control plane with BGP, EVPN, and BFD."

9.2 The 2-Minute Explanation

Add to the pitch: "The underlay uses eBGP with unique ASNs per leaf and a shared ASN for spines, following RFC 7938 — the same design Facebook and Microsoft use. EVPN provides control-plane MAC learning through Type-2 routes and inter-subnet routing through Type-5 routes, eliminating the flood-and-learn approach. The symmetric IRB model means each VTEP only needs its local subnets configured plus one shared L3VNI per VRF — the ingress leaf routes and encapsulates with the remote leaf's Router MAC, and the egress leaf routes locally. I had to handle Linux-specific requirements like disabling `rp_filter` for VXLAN, creating VRF routing tables, and understanding how FRR's Zebra daemon programs the kernel FDB and neighbor table. I also identified bugs in the init scripts — dead code after `exit 0` that made RMAC watchdog logic unreachable."

9.3 The Deep Technical Explanation for a Network Engineer Interviewer

Layer in: RMAC resolution mechanics (why FAILED neighbor entries break L3VNI), the specific EVPN NLRI format for Type-2 and Type-5, how `advertise-all-vni` works (Zebra scans kernel via netlink, sends VNI_ADD to bgpd), how BFD provides sub-second convergence by bypassing BGP hold timers, the difference between `bridge fdb` with `self` vs `master` flags, and how `ip rule` with `l3mdev-table` steers VRF lookups. You can walk through the complete h1→h2 packet journey from ARP through VXLAN encapsulation through decapsulation to h2.

9.4 Likely Interview Questions and Strong Answers

Q: What is VXLAN and why is it used?

Weak answer: "It is a tunneling protocol for virtual networks."

Strong answer: "VXLAN encapsulates complete L2 Ethernet frames in UDP/IP packets on port 4789 with a 24-bit VNI, extending L2 domains across L3 boundaries. It replaced VLANs in DC fabrics because VLANs are limited to 4,094 segments and cannot cross routers, while VXLAN supports 16.7 million segments and rides on any IP underlay with ECMP."

Q: What is the difference between L2VNI and L3VNI?

Weak answer: "L2 is for bridging, L3 is for routing."

Strong answer: "L2VNIs extend a single broadcast domain — one per subnet, carrying intra-subnet bridged traffic. L3VNIs provide inter-subnet routing transit — one per VRF/tenant, used in symmetric IRB where the VXLAN header carries the L3VNI and the inner destination MAC is the remote VTEP's Router MAC, signaling the egress leaf to route rather than bridge."

Q: Why does your lab use eBGP instead of OSPF?

Weak answer: "BGP is more scalable."

Strong answer: "Per RFC 7938, eBGP in DC eliminates OSPF's flooding domain problem — at scale, LSDB churn causes massive CPU overhead. BGP's path-vector nature limits propagation scope, AS_PATH provides natural loop prevention in Clos, and a single BGP session carries both underlay IPv4 and overlay EVPN routes. It is what Meta and Microsoft run in production."

Q: Walk me through what happens when h1 pings h2.

Use the complete symmetric IRB walkthrough from Section 5.1: h1 ARPs for the gateway, leaf1 routes in VRF, encapsulates with L3VNI and remote RMAC, underlay ECMP forwards, leaf2 decapsulates and routes to h2. The ability to trace a packet through every encapsulation and decapsulation step is the most technically impressive part of this project.

Q: What is rp_filter and why did you disable it?

Strong answer: "rp_filter is a Linux kernel security feature that drops packets whose source IP would not be reachable via the interface they arrived on. VXLAN requires disabling it because encapsulated packets arrive on physical uplinks with outer source IPs (remote VTEP loopbacks) that are reachable only via that same interface — not via a different path — and the asymmetric IRB forwarding model means the return path for a packet may differ from its arrival path. Linux takes the MAX of `conf/all/rp_filter` and the per-interface value, so both must be set to 0."

Q: What did you find when you audited the repository?

Strong answer: "I identified several issues: the most critical is that both leaf init scripts have an `exit 0` statement partway through, making everything after it dead code — including RMAC watchdog loops that were written to work around a real issue. The generated Containerlab state artifacts are committed to the repository and should be in `.gitignore`. The Route Target format uses per-leaf ASNs rather than a fixed admin value, which would not scale to more than two leaves without reconfiguring all existing nodes. The `vtysh.conf` is empty but still bound, adding noise without value."

9.5 What Is Most Technically Impressive

The project demonstrates three things interviewers value:

1. **Understanding of the full stack** from BGP control plane through Linux kernel dataplane — most candidates know one or the other, not both.
2. **Ability to identify bugs** (dead code, non-standard RTs, missing `.gitignore`, empty config file) rather than treating the lab as a black box.
3. **Lab-building initiative** using industry-standard open-source tools (FRR, Containerlab) rather than vendor simulators.

The ability to explain why `rp_filter=0` is necessary, or why a FAILED neighbor entry on `br50100` breaks L3VNI forwarding (the kernel cannot resolve RMAC to construct the inner Ethernet header), demonstrates depth that goes beyond configuration copy-paste.

10 Learning Roadmap

The following staged roadmap takes you from CCNA knowledge to confident operation and troubleshooting of an EVPN/VXLAN fabric. Each stage builds on the previous one.

Stage 1 — Fabric Fundamentals (Start Here)

Learn leaf-spine topology, why it replaced three-tier, ECMP, eBGP underlay design per RFC 7938, and loopback addresses as VTEP sources. Spend time drawing the topology on paper and tracing every BGP peering session. Deploy the `underlay.clab.yml` topology, verify eBGP sessions, and confirm ECMP from leaf1 to leaf2 via both spines.

Key commands to master: `show ip bgp summary`, `show ip route`, `ip route show`, `ip link show`.

Stage 2 — VXLAN Encapsulation

Understand the VXLAN packet format, UDP port 4789, 24-bit VNI, VTEP operation, and L2VNI bridging. Capture VXLAN traffic with Wireshark or `tcpdump` and verify the encapsulation stack. Add the VXLAN interface and bridge to leaf1 manually and ping across.

Key commands to master: `ip link add ... type vxlan`, `bridge fdb show`, `ip -s link show vxlanXXX`.

Stage 3 — EVPN Control Plane

Learn Type-2 routes for MAC/IP learning, Type-3 for BUM, ARP suppression, and BGP EVPN address family (AFI 25/SAFI 70). Enable `advertise-all-vni` and observe Type-2 routes populating in the EVPN RIB.

Key commands to master: `show bgp l2vpn evpn summary`, `show bgp l2vpn evpn route type 2`, `show evpn vni`.

Stage 4 — L3VNI and Inter-Subnet Routing

Master symmetric IRB, VRF, Type-5 routes, distributed anycast gateway, and RMAC. Add the L3VNI, configure `advertise ipv4 unicast`, and confirm that h1 can ping h2 across subnets.

Key commands to master: `show bgp l2vpn evpn route type 5`, `show bgp vrf vrf-ten1 ipv4 unicast`, `show evpn vni detail`.

Stage 5 — Linux Dataplane

Practice creating bridges, VXLAN interfaces, VRFs, reading FDB entries and neighbor tables, understanding VRF routing tables and `ip rule`, and disabling `rp_filter`. Deliberately break the neighbor table entry and observe how forwarding fails.

Key commands to master: `ip neigh show`, `bridge fdb show`, `ip route show table 10`, `ip rule show`, `sysctl -a | grep rp_filter`.

Stage 6 — FRR Architecture

Understand Zebra's role as kernel interface, bgpd EVPN implementation, how `advertise-all-vni` discovers VNIs, and how Zebra programs FDB/neighbor/routes via netlink. Read the FRR EVPN documentation and trace the ZAPI messages.

Resources: `https://docs.frrouting.org/en/latest/evpn.html`, FRR GitHub source code (`zebra/zebra_vxlan.c`).

Stage 7 — BFD and Convergence

Configure BFD, inject link failures, measure convergence times with ping probes, and understand how BFD notifies BGP. Run the dual-uplink failure scenario and quantify packet loss window.

Key commands to master: `show bfd peers`, `ip link set ethX down/up`, `date +"%H:%M:%S.%3N"`.

Stage 8 — Lab Automation and Troubleshooting

Build your own topologies with Containerlab, develop a systematic troubleshooting methodology (check control plane → check kernel state → check data plane), and automate verification scripts. Explore EVPN BGP Labs at `https://evpn.bgplabs.net/` for additional scenarios.

Concepts to revisit later: BGP route reflectors for EVPN, MLAG/ESI-based multi-homing (Type-1/Type-4 routes), EVPN with MPLS, VXLAN multicast flood-and-learn, cloud VTEP integration.

Table 9: Staged learning priority matrix

Stage	Topic	Focus	Priority
1	Leaf-spine + eBGP underlay	Deploy `underlay.clab.yml`; verify BGP	Essential
2	VXLAN encapsulation	Capture packets; verify FDB	Essential
3	EVPN control plane	Type-2 routes; ARP suppression	Essential
4	L3VNI + symmetric IRB	End-to-end h1→h2 ping	Essential
5	Linux dataplane	Break/fix neighbor table	Essential
6	FRR architecture	Read Zebra source; trace ZAPI	High
7	BFD + convergence	Measure recovery time	High
8	Lab automation	Containerlab; troubleshooting playbooks	Medium
Later	Multi-homing, MPLS, cloud VTEP	After stages 1–8 are solid	Defer

11 Reference Library

11.1 RFCs and Standards

Document / URL	Level	Why It Is Useful
RFC 7432 — BGP MPLS-Based Ethernet VPN `https://datatracker.ietf.org/doc/html/rfc7432`	Advanced	The core EVPN specification. Defines all route types (1–5), RD/RT semantics, MAC mobility, and multi-homing. Read Sections 7–9 for route type definitions.
RFC 8365 — NVO Solution Using EVPN `https://datatracker.ietf.org/doc/html/rfc8365`	Advanced	Extends EVPN to VXLAN encapsulation. Defines how EVPN NLRI maps to VXLAN VNIs in a Network Virtualization Overlay. Essential companion to RFC 7432.

Document / URL	Level	Why It Is Useful
RFC 7348 — VXLAN Specification `https://datatracker.ietf.org/doc/html/rfc7348`	Intermediate	Defines the VXLAN encapsulation format, UDP port 4789, and the 24-bit VNI. Start here for understanding the raw packet format.
RFC 5880 — BFD `https://datatracker.ietf.org/doc/html/rfc5880`	Intermediate	Full BFD specification. Explains the state machine, timer negotiation, and multiplier-based failure detection. Essential for understanding sub-second convergence.
RFC 7938 — BGP for DC Routing `https://www.rfc-editor.org/rfc/rfc7938.html`	Intermediate	Formal justification for using eBGP as the DC underlay routing protocol. Explains why BGP outperforms OSPF/IS-IS at scale in Clos fabrics.
RFC 9135 — Integrated Routing and Bridging in EVPN `https://www.rfc-editor.org/rfc/rfc9135`	Advanced	Defines symmetric and asymmetric IRB models formally. Required reading for understanding L3VNI design choices.
RFC 9136 — IP Prefix Advertisement in EVPN (Type-5) `https://www.rfc-editor.org/rfc/rfc9136`	Advanced	Defines the EVPN Type-5 route. Explains the RMAC extended community and how inter-subnet routing works at the prefix level.
RFC 8950 — Advertising IPv4 NLRI with IPv6 Next Hop `https://datatracker.ietf.org/doc/html/rfc8950`	Advanced	Enables BGP unnumbered (no IPv4 addresses on inter-switch links). FRR supports this natively.

Table 10: RFCs and IETF standards

11.2 FRRouting Documentation

Resource / URL	Level	Why It Is Useful
FRR EVPN Guide `https://docs.frrouting.org/en/latest/evpn.html`	Intermediate–Advanced	Official FRR EVPN documentation. Covers `advertise-all-vni`, L3VNI, VRF configuration, and all FRR-specific EVPN commands. The most authoritative reference for this project.
FRR BGP Guide `https://docs.frrouting.org/en/latest/bgp.html`	Intermediate–Advanced	Full BGP daemon documentation including all address-family configurations, BFD integration, and neighbor options.
FRR BFD Guide `https://docs.frrouting.org/en/latest/bfd.html`	Intermediate	BFD daemon configuration, timer parameters, and peer monitoring in FRR.

Resource / URL	Level	Why It Is Useful
FRR GitHub Source `https://github.com/FRRouting/frr`	Advanced	When documentation is ambiguous, read the source. `zebra/zebra_vxlan.c` explains how Zebra discovers VNIs and programs the kernel.

Table 11: FRRouting official documentation

11.3 Containerlab

Resource / URL	Level	Why It Is Useful
Containerlab main site `https://containerlab.dev/`	Beginner	Start here. Documentation covers installation, topology files, node kinds, binds, exec, and sysctls.
Topology definition reference `https://containerlab.dev/manual/topo-def-file/`	Beginner–Intermediate	Complete reference for the `.clab.yml` syntax used in this project.
Lab examples `https://containerlab.dev/lab-examples/lab-examples/`	Intermediate	Ready-made topologies including FRR, Nokia SR Linux, Arista cEOS. Good for comparison and learning.

Table 12: Containerlab documentation and resources

11.4 Vendor Design Guides

Resource / URL	Level	Why It Is Useful
NVIDIA Cumulus EVPN Docs `https://docs.nvidia.com/networking-ethernet-software/cumulus-linux-59/Network-Virtualization/Ethernet-Virtual-Private-Network-EVPN/`	Intermediate–Advanced	Cumulus Linux uses FRR internally. This documentation mirrors real-world FRR EVPN deployment closely. Especially useful for L3VNI and RMAC troubleshooting.
Arista EVPN Configuration Guide `https://www.arista.com/en/um-eos/eos-evpn-overview`	Intermediate–Advanced	Arista EOS is widely deployed. Shows production-grade EVPN/VXLAN config. Useful for understanding vendor-specific differences from the open-source FRR approach.

Resource / URL	Level	Why It Is Useful
Cisco VXLAN BGP EVPN Design Guide `https://www.cisco.com/c/en/us/td/docs/dcn/whitepapers/cisco-vxlan-bgp-evpn-design-and-implementation-guide.html`	Intermediate–Advanced	Comprehensive Cisco implementation guide. Excellent for understanding symmetric IRB, anycast gateway, and multi-site EVPN even if the CLI differs from FRR.
VXLAN: BGP EVPN with FRR (Vincent Bernat) `https://vincent.bernat.ch/en/blog/2017-vxlan-bgp-evpn`	Intermediate	Practical FRR-based EVPN/VXLAN walkthrough by a well-respected Linux networking engineer. Explains Linux bridge and FDB behavior in the context of EVPN.

Table 13: Vendor and practitioner design guides

11.5 Learning Resources and Courses

Resource / URL	Level	Why It Is Useful
NetworkLessons.com VXLAN Hub `https://networklessons.com/vxlan`	Beginner–Intermediate	Comprehensive VXLAN and EVPN article series with diagrams and step-by-step examples. Start with the VXLAN introduction, then L2VNI, then L3VNI.
NetworkLessons L2VNI Tutorial `https://networklessons.com/vxlan/vxlan-mp-bgp-evpn-l2-vni`	Beginner	Focused L2VNI walkthrough. Explains Type-2 route generation and ARP suppression with clear packet diagrams.
NetworkLessons L3VNI Tutorial `https://networklessons.com/vxlan/vxlan-mp-bgp-evpn-l3-vni`	Intermediate	Deep dive into L3VNI, inter-VNI routing, and EVPN Type-5 behavior. Directly relevant to this project.
ipSpace.net EVPN Deep Dive `https://www.ipspace.net/EVPN_Technical_Deep_Dive`	Advanced	Ivan Pepelnjak's most thorough EVPN treatment. Covers design trade-offs, multi-homing, route reflectors, and scaling considerations that go beyond basic labs.
ipSpace.net EVPN BGP Labs `https://evpn.bgplabs.net/`	Intermediate–Advanced	Hands-on lab exercises for EVPN built on Containerlab. Excellent progression from underlay through Type-5 to multi-site.

Resource / URL	Level	Why It Is Useful
ipSpace.net Blog `https://blog.ipspace.net/`	Intermediate–Advanced	Ivan Pepelnjak's blog. Rigorous analysis of real-world DC networking decisions. Use the EVPN tag for directly relevant posts.
Packet Pushers HN696: EVPN Fundamentals `https://packetpushers.net/podcasts/heavy-networking/HN696-evpn-fundamentals-and-some-vxlan-with-tony-bourke/`	Beginner	Audio deep-dive on EVPN fundamentals, accessible and well-paced. Good for reinforcing concepts during commute or exercise.
Jeremy's IT Lab LAN Architectures `https://www.youtube.com/watch?v=PvyEcLhmNBk`	Beginner	CCNA-level LAN architecture video including spine-leaf basics. Perfect starting point if three-tier vs leaf-spine is still abstract.

Table 14: Online learning resources

11.6 Linux Networking Documentation

Resource / URL	Level	Why It Is Useful
Linux Kernel VRF Documentation `https://docs.kernel.org/networking/vrf.html`	Intermediate–Advanced	Official kernel docs for VRF devices, routing table association, and the `l3mdev` framework. Essential for understanding `ip link add type vrf table N`.
Linux Kernel Bridge Documentation `https://docs.kernel.org/networking/bridge.html`	Intermediate	How Linux bridges work, FDB management, and VXLAN-to-bridge integration at the kernel level.
`bridge(8)` Man Page `https://www.man7.org/linux/man-pages/man8/bridge.8.html`	Intermediate	Complete reference for the `bridge` command. Covers `fdb show/add/replace`, `link show`, and `neigh_suppress` flag.
`ip-vrf(8)` Man Page `https://www.man7.org/linux/man-pages/man8/ip-vrf.8.html`	Intermediate	Reference for `ip vrf exec` and VRF-aware command execution, useful for testing in-VRF routing decisions.
Red Hat VRF Overview `https://developers.redhat.com/articles/2025/01/08/overview-virtual-routing-and-forwarding-vrf-linux`	Intermediate	Readable introduction to Linux VRF for engineers coming from a hardware/Cisco background. Explains VRF routing table mechanics in accessible language.

Table 15: Linux networking documentation

11.7 Related Open-Source Labs

Resource / URL	**Level**	**Why It Is Useful**
Multivendor EVPN Lab (Nokia SR Linux) `https://github.com/srl-labs/multivendor-evpn-lab`	Intermediate	Containerlab-based multivendor EVPN topology. Useful for comparing FRR behavior against Nokia SR Linux in the same fabric.
FRR VXLAN Labs (Martimy) `https://github.com/martimy/clab_vxlan_frr`	Intermediate	Additional FRR-based VXLAN Containerlab examples with progressive complexity.
FRR EVPN-VXLAN Labs (dbono711) `https://github.com/dbono711/clab-frr-evpn-vxlan`	Intermediate	Another FRR EVPN Containerlab implementation. Useful for cross-referencing configuration choices with this project.

Table 16: Related open-source Containerlab EVPN labs